**LLM Comp90024 Group36**

## LLM Usage in Bluesky Fission Deployment and Environment Configuration —Xinyao Li 1560643

Due to unclear error logs, deployment issues were difficult to diagnose. During deployment and testing, every configuration step had to be flawless; any errors requiring redeployment could lead to cluster chaos or conflicts. Therefore, I used ChatGPT to check YAML configuration issues, key mounting behaviour, and function environment settings. These discussions primarily helped identify potential causes and narrow down the debugging direction, while the final deployment settings and configuration changes were done manually during implementation.

### I. Fission Timeout Behaviour

When initially writing the Bluesky crawler, I simply encapsulated a traditional full-data crawler code into a Fission function. However, during the testing process, I found that this method was not suitable for Fission. When running `fission fn test`, due to the long execution time, the function often timed out, making it difficult to observe the actual crawled data volume continuously and to determine whether the problem lay in the code logic or Fission itself. At first, I thought it was just that the default timeout was too short, so I tried increasing the test time limit from the default value to 5 minutes. But the historical data crawling still couldn't be completed stably. I began to suspect that Fission was not suitable for directly running traditional long-running full-data crawlers.

I consulted ChatGPT to understand the differences between traditional crawlers and Fission timer-triggered functions, and discussed whether the timeout in `fission fn test` necessarily meant that the timer execution would fail. Through these discussions, I gradually realised that Fission function tests are more similar to synchronous tests and are prone to execution time limits, whereas the execution mode triggered by the timer is better suited to splitting tasks and running them multiple times. These discussions also made me start to rethink the execution method of the crawler, rather than continuing to try to solve the problem by increasing the timeout time.

### II. Incremental Crawler Logic Design

After repeatedly discussing with ChatGPT, I gradually realised that if the crawler wants to continuously run under Fission's timer-triggered execution, it needs to save the crawling progress separately instead of relying on a single execution to complete the capture of all historical data. I also asked about how to manage a large number of queries in the serverless crawler, how to distinguish between real-time crawl and historical crawl, and how the historical data should be gradually advanced instead of being captured all at once. These discussions led me to gradually incorporate the cursor-based incremental crawling logic into the crawler design. Finally, I used Elasticsearch to save the crawling progress, including historical boundaries, last_seen_time, query rotation status, and whether a certain query has completed the historical capture, enabling the crawler to gradually advance in multiple timer-triggered executions instead of starting each time anew.

### III. Kubernetes Secret and Login Issues

Later during development, the shared Kubernetes es-secret configuration was reset while updating the database configuration, which caused the Bluesky account credentials to become empty. As a result, the crawler could no longer log in successfully and stopped collecting data.

At first, I suspected the issue might be related to the Bluesky API or Elasticsearch connectivity, so I used ChatGPT to help check possible causes, including Kubernetes secret mounting paths, secret values, and environment variable loading behaviour.

After checking these areas step by step, I identified that the stored credentials were missing from the secret configuration. I then restored the secret values and added additional validation checks to the login and Elasticsearch connection logic to make similar deployment issues easier to debug later.

## LLM Usage in FuelWatch Pipeline and Analytics Development —— Hongkun Zhang 1638502

During the development of the FuelWatch-related components, ChatGPT was mainly used as a discussion and debugging assistant to help reason through problems encountered during the implementation process. Similar to the Bluesky crawler development, many of the issues were not simple coding mistakes, but instead came from understanding how different parts of the cloud-based workflow interacted together under Kubernetes, Fission, and Elasticsearch.

### I. FuelWatch Data Transformation and Aggregation Design

When first working with the FuelWatch historical datasets, I initially attempted to directly clean and store the raw station-level records into Elasticsearch. However, during notebook development and testing, I gradually realised that the raw data contained multiple fuel categories, duplicated entries, and inconsistent update timing, making the later analytics difficult to interpret.

I discussed with ChatGPT different approaches for structuring the fuel pipeline, including whether the project should preserve all station-level records or simplify the data into representative daily statistics that could later be aligned with Reddit and Bluesky discussion timelines.

Through these discussions, I gradually redesigned the workflow into a separate intermediate aggregation stage, where one representative daily ULP fuel price was generated from the raw FuelWatch records. This aggregated dataset mainly served as an internal reference dataset during development and validation. The final notebook analytics and front-end visualisations instead used a further integrated daily dataset produced through the backend and Elasticsearch workflow.

### II. Fission Timer Workflow and Delayed Official Data Updates

ChatGPT discussions played an important role in deploying the FuelWatch workflow to the timer-triggered Fission function.

Initially, I configured the FuelWatch aggregation timer to run once a day at 2 am, because I expected the official daily fuel data to be fully available by then. However, during front-end testing and Elasticsearch validation, I gradually noticed that there were sometimes incomplete aggregation results, including the case where the calculated daily fuel price unexpectedly turned to 0.

At first, I suspected that the problem might be related to an Elasticsearch write or aggregation logic error. However, after cross-examining the pipeline and discussing the workflow behaviour with ChatGPT, I gradually realised that the problem was more closely related to the update timing of the official FuelWatch source data itself. In some cases, planned aggregation tasks were performed before official records were fully released, resulting in incomplete daily data being processed into intermediate datasets.

These discussions gradually shifted my attention from debugging the aggregation code itself to thinking about the interaction between upstream official data availability, Fission timer scheduling, and downstream Elasticsearch analysis behaviour.

So, I redesigned the timer workflow so that the aggregation process executes every six hours instead of relying on a single execution once a day. This reduces the risk of incomplete daily updates affecting our analytics pipeline and makes our frontend visualisations more stable.

These discussions also helped me better understand the differences between timer-triggered serverless workflows and traditional manually executed scripts, especially when the upstream data sources themselves update asynchronously and are not guaranteed to follow a fixed time.

### III. Limitations encountered during LLM-assisted debugging

During many debugging sessions, we generated Ubuntu shell commands and Elasticsearch query commands that seemed logically correct, but they failed to execute because the command syntax, format, or runtime assumptions did not exactly match the actual project environment. As development progressed, I saw it as a tool to narrow down the causes of failures, reason about workflow behaviour, and explore alternative implementations during debugging, which helped me better understand and explore uncharted territories.

## LLM Usage in Elasticsearch Infrastructure and Backend API Development — Hanyue Li 1650115

During the development of the Elasticsearch infrastructure and backend API layer, Claude and Gemini were primarily used as writing and review assistants. They were also utilized as architectural sounding boards and debugging assistants to optimize data layer performance and ensure API contract reliability.

**I. Compute Pushdown and Memory Guard Design**

Our raw fuel database contained over 4.35 million records. The initial prototype design ran synchronous ES|QL queries to truncate timestamps and calculate daily national averages. However, testing this approach against the live cluster caused significant execution latency and threatened to hit the Fission container memory ceiling.

We used the LLM to evaluate distributed computing trade-offs within Elasticsearch. These discussions guided our decision to pivot to a Compute Pushdown Strategy using a dedicated, sharded summary index (fuel-daily-summary). By leveraging the native Elasticsearch Aggregations API with size=0, the heavy math was pushed directly down to the cluster's 3 primary shards. The Python backend then only downloaded a lightweight summary of ~1,500 rows. This design insulated the serverless API layer from out-of-memory crashes and reduced the frontend data delivery overhead down to O(1) lookups.

**II. Overcoming Non-Relational Join Barriers**

A significant obstacle appeared when we attempted to correlate raw fuel timelines with social media sentiment scores. Elasticsearch operates as a NoSQL engine and does not natively support strict SQL-style relational joins. Our early attempts to use database-layer lookups and ENRICH policies failed because the engine could only return the first matched document it encountered. This introduced severe statistical inaccuracies to our average price calculations.

We used the LLM to reason through alternative multi-index orchestration models. Through these sessions, we abandoned cluster-layer joining entirely. We designed an Application-Layer Hash-Map Merge instead. The API fetched decoupled, pre-aggregated payloads from both indices independently. The backend Python script then executed a clean, memory-safe merge in-memory. This architectural change successfully aligned the timeline data without running into NoSQL structural limitations.

**III. Resolving Schema-Mapping Mismatches**

During data source validation, we discovered that automatic dynamic mapping had indexed numeric fields—like sentiment_score and product_price—as text instead of float. In a legacy relational database, fixing a schema mismatch requires a slow, blocking alter table command. Our team could not afford a full reindex of 4.3 million records during active local testing.

We consulted AI to explore schema-resilient aggregation strategies. The LLM assisted in drafting native JSON query bodies that explicitly targeted raw string fields using a missing value handler. Because writing raw Elasticsearch Query DSL syntax is highly complex and unintuitive compared to standard SQL, we relied heavily on Kibana’s "Translate to Query DSL" utility. This feature automatically converted our experimental ES|QL lines into structured JSON blocks, which we smoothly integrated directly into our FastAPI backend. This allowed the engine to bucket data accurately without enabling memory-intensive field data, preserving database stability.

## LLM Usage in Pipeline Development and Reference Search — Yichen Sun 1682096

### I. CI/CD Pipeline Development

Large language models (Claude) were used at several points during the project. For the CI/CD pipeline, the main use was understanding unfamiliar tooling. This included GitLab pipeline syntax and how needs controls job parallelism. Pytest module stubbing was also unfamiliar — specifically, how to stub out elasticsearch8 before importing project code in a CI environment. flake8 and pip-audit integration was clarified the same way. The pipeline configuration and test logic were written by the developer directly. LLMs were not used to generate code.

I started with a minimal CI configuration.It's a single test job that checks Python version, directory structure, and key file existence. It worked, but it was not testing anything meaningful about the actual code.

The first real challenge came when building the deploy stage. The initial approach called deploy.sh, which ran fission package update internally. This failed with a "package is used by multiple functions" error. Adding --force to the script fixed the error message, but the flag was not being picked up reliably — likely a version-specific issue with the Fission CLI. I abandoned deploy.sh entirely and wrote the deployment commands directly into .gitlab-ci.yml, which gave me full control over the flags. This also made the pipeline more transparent, since the exact commands being run were visible in the CI configuration rather than buried in a shell script.

The next issue was a file path mismatch. The zip step moved reddit.zip two directories up with mv reddit.zip ../../, which placed it under backend/ — but the fission package update command was looking for it at backend/fission/reddit.zip. Fixing the path resolved the package update, but then fission fn update also started failing until -f was added to every function update command as well.

Once the deploy stage was stable, I expanded the CI stage with meaningful unit tests. Writing these introduced a YAML problem: the after_script block used a | block scalar, which GitLab's YAML parser rejected. I simplified that section, but the pipeline still failed. The actual cause turned out to be CRLF line endings — Windows had converted the line endings when the file was saved locally, and this corrupted some of the multi-line YAML structures in the deploy script block. I rewrote the entire file using only simple single-line list items, avoiding all block scalars and folded scalars, which resolved the parsing errors.

The unit tests themselves required stubbing out elasticsearch8 and the platform-specific crawler modules before Python resolved imports, since neither is available in a CI runner. I used sys.modules to inject mock objects at the top of each test file. The final test suite runs 23 tests across the Reddit and Bluesky harvesters, covering secret fallback, document skip logic, error counting, cursor retrieval under failure, query batch rotation, and exception handling.

### II. Reference Search

Finding relevant references was another area where LLMs helped. The project spans a few different fields — cloud infrastructure, social media data collection, and sentiment analysis. It was not always obvious which search terms would surface useful papers. Claude was consulted to suggest starting points. Papers were then located through Google Scholar and read independently. No paper content was summarised or interpreted by the LLM.

## LLM Usage in Reddit Fission Deployment and Data Collection — Junyao Zhang

During the development of the Reddit data harvesting pipeline, Claude was primarily used as a step-by-step deployment guide and debugging assistant. Unlike traditional development workflows, deploying Fission functions on a live Kubernetes cluster required every configuration step to be precise, as errors could cause function failures, pod specialisation issues, or data loss. Claude was consulted throughout the entire process to provide real-time guidance, diagnose error messages, and suggest corrective actions.

**I. Fission Package Build and Deployment Issues**

During the initial deployment of the Reddit Fission package, several consecutive build failures occurred that were difficult to diagnose from error messages alone. These included a permission denied error caused by missing executable permissions on build.sh, incorrect pip install syntax in the build script that did not match the Fission builder environment, a missing __init__.py file that prevented Python from recognising the function directory as a package, and an incorrect elasticsearch library name that caused import failures at runtime. Claude was consulted to identify the root cause of each error by interpreting the Fission build logs and comparing the configuration against the teacher's reference implementation. The corrective actions, including chmod, repackaging, and updating the import statements, were performed manually by the developer.

**II. Historical Data Collection via Arctic Shift API**

When attempting to collect historical Reddit data from 2022 onwards, the Reddit public JSON API proved insufficient, as it only returns the most recent approximately 1,000 posts per subreddit. Claude was consulted to identify alternative data sources, which led to the discovery of the Arctic Shift API, a community-maintained Reddit archive covering 2005 to the present. Claude also assisted in diagnosing API parameter errors, including incorrect date format (YYYY-MM-DD instead of Unix timestamps), unsupported sort parameter values, and request failures caused by overly large time ranges. The solution of querying data month by month was implemented by the developer after identifying the root cause through error analysis.

**III. Debugging Kubernetes Secret Loss**

During the final days before submission, the shared es-secret Kubernetes Secret was accidentally overwritten by a teammate, leaving only a single password field instead of the original six fields. This caused all Fission functions to fail silently, as the ES_HOST field was missing and the ElasticSearch client could not initialise. Claude was consulted to systematically diagnose the issue by checking the timer status, testing the function manually, and inspecting the secret contents. Once the root cause was identified, the secret was reconstructed manually using kubectl with the correct field values, and all functions resumed normal operation.

Throughout the development process, all deployment commands, configuration changes, and code modifications were written and executed by the developer. Claude served as a diagnostic and advisory tool, helping interpret error messages, suggest debugging steps, and explain underlying concepts, while the final implementation decisions and actions remained with the developer.